

The logo of the University of Wah is located at the top of the page. It features a blue header with a white curved line and the text "University of Wah" in blue.

University of Wah

LAB MANUAL

Computer Architecture

LAB 02

Saba Latif

Saba.latif@uow.edu.pk

QtSpim Interface Explanation

1. Left Side — Registers Window

This shows **all CPU registers** of the **MIPS processor**.

Registers are small memory locations inside the CPU used for fast data storage during program execution.

Column	Meaning	Example
PC	Program Counter – shows address of the next instruction to execute.	PC = 0
EPC, Cause, BadVAddr, Status	System control registers used for exceptions (advanced topic).	—
HI and LO	Store results of multiplication/division instructions.	HI = 0, LO = 0
R0–R16	General-purpose registers for data and addresses.	Example: R4 [a0] = 2

Registers have both **number (R0–R31)** and **name (like a0, t0, v0)** for convenience:

- a0–a3: arguments (used to pass data to functions)
- t0–t9: temporary registers
- v0–v1: values (used to return function results)
- s0–s7: saved registers
- sp: stack pointer (R29)
- ra: return address (R31)

2. Right Side — Data Segment / Text Segment Tabs

There are tabs like:

- **Data**
- **Text**

Tab	Meaning	Example
Text	Contains machine instructions (your assembly program code).	When you load a program, this shows .text section.
Data	Shows variables, constants, and user data from .data section of your program.	Shows “User data segment [10000000]...”
User Stack	Part of memory used by functions and temporary storage (local variables, return addresses).	You can see many hexadecimal numbers and some readable text (like hello.asm, USERNAME, etc.) — these are stored in memory.

3. Bottom Section — Console / Log Area

This section shows **messages, program output, and status info**.

It displays:

- Version info of SPIM (SPIM Version 9.1.24 of August 1, 2023)
- Messages like “Memory and registers cleared”
- Any **runtime errors, syscall outputs, or program results**

When you run your assembly program, if it prints text using syscall, the output will appear here.

Example Flow (How It Works Together)

1. You **load** your assembly file (e.g., hello.asm) from File → Load File.
 2. The **Text Segment** shows your instructions.
 3. The **Data Segment** shows your declared variables.
 4. The **Registers window** shows the current CPU register values.
 5. When you **run** the program:
 - Registers change values.
 - Memory in Data/Stack sections updates.
- Output or messages appear in the **bottom window**.

What Are Registers?

- Registers are **small memory boxes inside the CPU**. Each register stores a number (like data, address, or result of a calculation).
- MIPS processor has **32 general-purpose registers**, named:
\$0, \$1, \$2, ... \$31
or symbolically as:
\$zero, \$at, \$v0, \$a0, \$t0, \$s0, \$sp, \$ra, etc.

Register Groups and Their Uses:

Register Name	Number	Meaning	Easy Explanation	Example
\$a0–\$a3	4–7	Argument registers	These carry input data (arguments) when calling a function.	If you want to print a number, you put it in \$a0 before calling a syscall or function.
\$v0–\$v1	2–3	Value registers	These hold function results or return values. Also used to tell which system call to execute.	Example: \$v0 = 1 means “print integer” syscall. After a function ends, the result is placed in \$v0.
\$t0–\$t9	8–15, 24–25	Temporary registers	Used to hold temporary values during calculations. CPU can freely use them — not saved after a function call.	Example: you do addition and store result temporarily in \$t0.
\$s0–\$s7	16–23	Saved registers	Used to store data that must remain unchanged after a function call. The program saves and restores them manually.	Example: if you store a loop counter or variable in \$s0, it stays safe during function calls.
\$sp	29	Stack Pointer	Points to the top of the stack (special memory	When a function starts, \$sp moves

			area used to save data temporarily). The stack grows downward (toward smaller addresses).	down to make space; when it ends, \$sp moves up again.
\$ra	31	Return Address	When a function (or syscall) is called, the address to return to is stored in \$ra.	Example: after finishing a subroutine, the CPU jumps back to the address in \$ra.
\$at	1	(Assembler Temporary)	Memory address temporarily stored by assembler.	Used when assembler expands pseudo-instructions like <code>la</code> , <code>li</code> , etc.

Example:

main:

```
li $a0, 5    # load 5 in argument register a0
li $a1, 3    # load 3 in argument register a1
jal add_nums # call function (jump and link)
move $t0, $v0 # store result from v0 in t0
li $v0, 10   # exit
syscall
```

add_nums:

```
add $v0, $a0, $a1 # v0 = a0 + a1
jr $ra           # return to address in ra
```

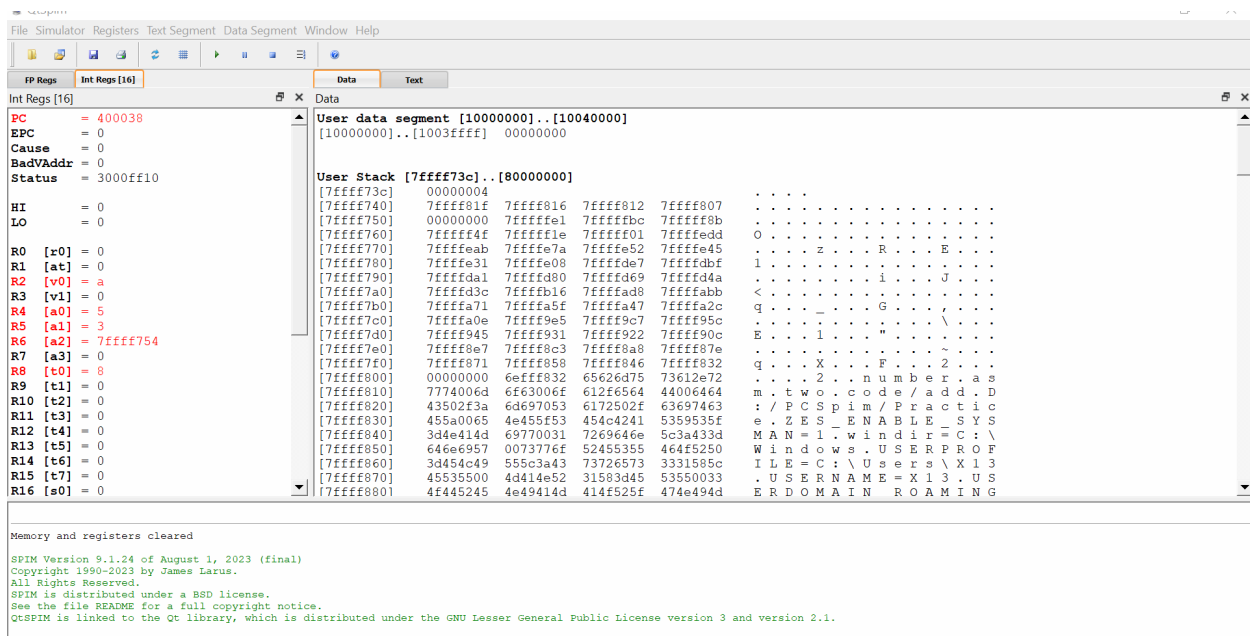
What Happens Step-by-Step:

Step	Action	Registers Used
1	\$a0 = 5, \$a1 = 3 → Arguments ready	a0–a3
2	jal add_nums → program jumps to function and \$ra stores return address	ra
3	Function runs: \$v0 = \$a0 + \$a1 → result (8) stored in \$v0	v0
4	jr \$ra → jumps back to main using \$ra value	ra
5	move \$t0, \$v0 → copy result into temporary register	t0

Final: \$t0 has **8**, the sum of 5 + 3.

Type	Registers	Used For	Example
Argument	\$a0–\$a3	Pass input to functions	\$a0 = 5
Return Value	\$v0–\$v1	Store output from functions	\$v0 = 8
Temporary	\$t0–\$t9	Intermediate calculations	\$t1 = \$t0 + 4
Saved	\$s0–\$s7	Preserve data	\$s0 = loop counter
Stack Pointer	\$sp	Manage stack memory	\$sp = \$sp - 4
Return Address	\$ra	Remember where to return after function	jr \$ra

Output:



Write a MIPS program to add two numbers (e.g., 5 and 10) and display their sum on the screen.

```
.data
    msg: .asciiz "The sum is: "    # Message to display
.text
.globl main
main:
    # Load numbers into registers
    li $t0, 5                      # Load 5 into $t0
    li $t1, 10                     # Load 10 into $t1
    # Add the numbers
    add $t2, $t0, $t1              # $t2 = $t0 + $t1
    # Print message
    li $v0, 4                      # Syscall code for print string
    la $a0, msg                    # Load address of msg into $a0
    syscall
    # Print sum
    li $v0, 1                      # Syscall code for print integer
    move $a0, $t2                  # Move sum into $a0 for printing
    syscall
```

Exit program

li \$v0, 10

Syscall for exit

syscall

Output:

The screenshot displays the SPIM MIPS simulator interface. The 'Int Regs [16]' window shows the following register values:

Register	Value
PC	40004c
EPC	0
Cause	0
BadVAddr	0
Status	3000ff10
HI	0
LO	0
R0 [r0]	0
R1 [at]	0
R2 [v0]	a
R3 [v1]	0
R4 [a0]	f
R5 [a1]	7ffff708
R6 [a2]	7ffff734
R7 [a3]	0
R8 [t0]	5
R9 [t1]	a
R10 [t2]	f
R11 [t3]	0
R12 [t4]	0
R13 [t5]	0
R14 [t6]	0
R15 [t7]	0
R16 [s0]	0

The 'Data' window shows memory addresses and their corresponding values:

Address	Value
100000	[100000]
100100	[100100]
100100	[100100]

The 'Console' window displays the output of the program:

```
The sum is: 15
```

The bottom status bar indicates: 'Memory and registers cleared'.

QtSpim

File Simulator Registers Text Se...

FP Regs Int Regs [16]

Int Regs [16]

PC	=	40004c
EPC	=	0
Cause	=	0
BadVAddr	=	0
Status	=	3000ff10
HI	=	0
LO	=	0
R0 [r0]	=	0
R1 [at]	=	0
R2 [v0]	=	a
R3 [v1]	=	0
R4 [a0]	=	f
R5 [a1]	=	7ffff708
R6 [a2]	=	7ffff734
R7 [a3]	=	0
R8 [t0]	=	5
R9 [t1]	=	a
R10 [t2]	=	f
R11 [t3]	=	0
R12 [t4]	=	0
R13 [t5]	=	0
R14 [t6]	=	0
R15 [t7]	=	0
R16 [s0]	=	0

→ Stored a=10 of hexadecimal numbers for program termination code of syscall

→ Stored results f=15 because print function always get value from \$a0 register.

→ Temporary registers store the integers 5 and 10 to perform addition.

→ Temporary register store the result of sum 15= f

Write a MIPS program that take two numbers as input from the user and print their sum.

.data

prompt1: .asciiz "Enter first number: "

prompt2: .asciiz "Enter second number: "

resultMsg: .asciiz "The sum is: "

.text

.globl main

main:

--- Ask for first number ---

li \$v0, 4 **# Print string syscall**

la \$a0, prompt1 **# Load prompt1 message**

syscall

li \$v0, 5 **# Read integer syscall**

syscall

move \$t0, \$v0 **# Store first number in \$t0**

--- Ask for second number ---

```
# --- Add the numbers ---
add $t2, $t0, $t1    # $t2 = $t0 + $t1
# --- Display the result ---
li $v0, 4             # Print "The sum is: "
la $a0, resultMsg
syscall
li $v0, 1             # Print integer syscall
move $a0, $t2         # Move sum into $a0
syscall
# --- Exit program ---
li $v0, 10            # Exit syscall
syscall
```

Registers used in this program:

Register	Purpose
\$v0	Used for syscall codes and return values
\$a0	Argument register (for print/read syscalls)
\$t0	Stores first number
\$t1	Stores second number
\$t2	Stores the sum

Basic MIPS System Call Codes:

Code (\$v0)	Service Name	Description	Arguments (Input Registers)	Return Value (Output)
1	Print Integer	Prints an integer	\$a0 = integer	—
2	Print Float	Prints a float	\$f12 = float	—
3	Print Double	Prints a double	\$f12 = double	—
4	Print String	Prints a null-terminated string	\$a0 = address of string	—
5	Read Integer	Reads an integer from user	—	\$v0 = integer
6	Read Float	Reads a float from user	—	\$f0 = float
7	Read Double	Reads a double from user	—	\$f0 = double
8	Read String	Reads a string (input text)	\$a0 = buffer address, \$a1 = length	—
9	Allocate Memory	Allocates heap memory (like malloc)	\$a0 = number of bytes	\$v0 = address
10	Exit	Ends the program	—	—
11	Print Character	Prints a single character	\$a0 = ASCII value	—
12	Read Character	Reads one character from input	—	\$v0 = ASCII value
13	Open File	Opens a file	\$a0 = filename addr,	\$v0 = file

			\$a1 = flags, \$a2 = mode	descriptor
14	Read File	Reads data from a file	\$a0 = file descriptor, \$a1 = buffer, \$a2 = length	\$v0 = number of bytes read
15	Write File	Writes data to a file	\$a0 = file descriptor, \$a1 = buffer, \$a2 = length	\$v0 = number of bytes written
16	Close File	Closes an open file	\$a0 = file descriptor	—
17	Exit with Code	Terminates program with a specific exit code	\$a0 = exit code	—

Declaring Variables in QtSpim

- Variables are declared in the **.data** segment of a MIPS program.

Example:

```
.data
num1: .word 5      # integer variable with initial value 5
num2: .word 10     # another variable
result: .word 0    # variable to store result

.text

.globl main
main:
    lw $t0, num1    # load num1 into register $t0
    lw $t1, num2    # load num2 into register $t1
    add $t2, $t0, $t1 # result = num1 + num2
    sw $t2, result  # store result back into memory
```

```
li $v0, 10          # exit program
```

```
syscall
```

You can access them using **load (lw)** and **store (sw)** instructions.

Directive	Meaning	Bytes reserved
.byte	Store 8-bit value	1 byte
.half	Store 16-bit value	2 bytes
.word	Store 32-bit value	4 bytes
.asciiz	Store string (null-terminated)	depends on string length

Q.1.Declare three variables x, y, and result.

Initialize x = 8, y = 3.

Perform the following operations and display the results:

- Sum
- Difference
- Product

```
.data
```

```
x:.word 25    # variable x = 25
```

```

y:.word 17    # variable y = 17
sum:.word 0   # to store x + y
diff:.word 0  # to store x - y
prod:.word 0  # to store x * y
msg1:.asciiz "Sum = "
msg2:.asciiz "\nDifference = "
msg3:.asciiz "\nProduct = "

.text
.globl main
main:
    lw $t0, x      # $t0 = x (25)
    lw $t1, y      # $t1 = y (17)

    #-----
    # Perform SUM
    #-----

    add $t2, $t0, $t1 # t2 = x + y
    sw $t2, sum      # store result in memory

    # Print "Sum = "
    li $v0, 4

```

```
la $a0, msg1
```

```
syscall
```

```
# Print sum value
```

```
li $v0, 1
```

```
lw $a0, sum
```

```
syscall
```

```
#-----
```

```
# Perform DIFFERENCE
```

```
#-----
```

```
sub $t3, $t0, $t1 # t3 = x - y
```

```
sw $t3, diff      # store result
```

```
# Print "\nDifference = "
```

```
li $v0, 4
```

```
la $a0, msg2
```

```
syscall
```

```
# Print difference value
```

```
li $v0, 1
```

```
lw $a0, diff
```

```
syscall
```

```
#-----
```

```
# Perform PRODUCT
```

```
#-----
```

```
mul $t4, $t0, $t1 # t4 = x * y
```

```
sw $t4, prod      # store result
```

```
# Print "\nProduct = "
```

```
li $v0, 4
```

```
la $a0, msg3
```

```
syscall
```

```
# Print product value
```

```
li $v0, 1
```

```
lw $a0, prod
```

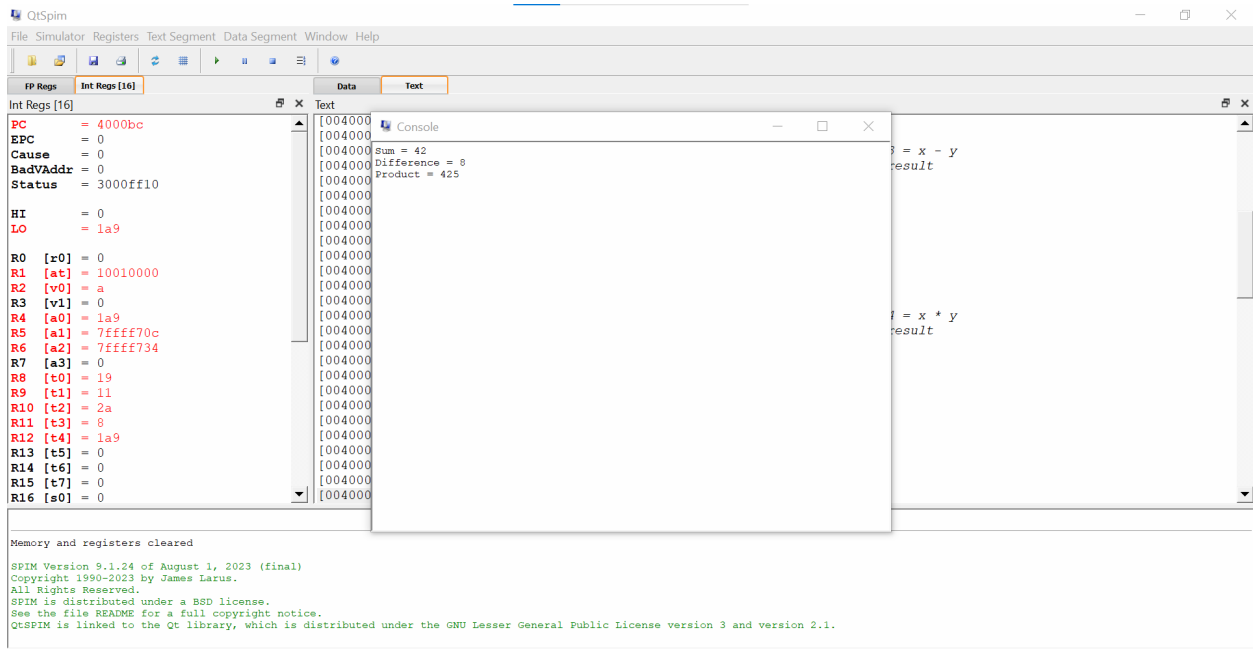
```
syscall
```

```
# Exit program
```

```
li $v0, 10
```

```
syscall
```

output:



Q.2. Declare two integer variables $x = 10$ and $y = 20$. Swap their values using temporary registers, store the swapped values back to memory, and display the before and final results after swapping.

.data

x: .word 10 # variable x = 10

y: .word 20 # variable y = 20

msg1: .asciiz "Before swapping:\n"

msg2: .asciiz "x = "

msg3: .asciiz "\ny = "

msg4: .asciiz "\n\nAfter swapping:\n"

.text

.globl main

main:

Display before swapping

li \$v0, 4

la \$a0, msg1

syscall

li \$v0, 4

la \$a0, msg2

syscall

li \$v0, 1

lw \$a0, x

syscall

li \$v0, 4

la \$a0, msg3

syscall

li \$v0, 1

lw \$a0, y

syscall

Load variables

lw \$t0, x # \$t0 = x (10)

lw \$t1, y # \$t1 = y (20)

Swap using a temporary register \$t2

move \$t2, \$t0 # t2 = x

```
move $t0, $t1    # t0 = y
move $t1, $t2    # t1 = x
# Store swapped values back to memory
    sw $t0, x
sw $t1, y
# Display after swapping
li $v0, 4
la $a0, msg4
syscall
li $v0, 4
la $a0, msg2
syscall
li $v0, 1
lw $a0, x
syscall
li $v0, 4
la $a0, msg3
syscall
li $v0, 1
lw $a0, y
syscall
# Exit program
```

syscall

Output:

Recursion in MIPS

Every recursive function must:

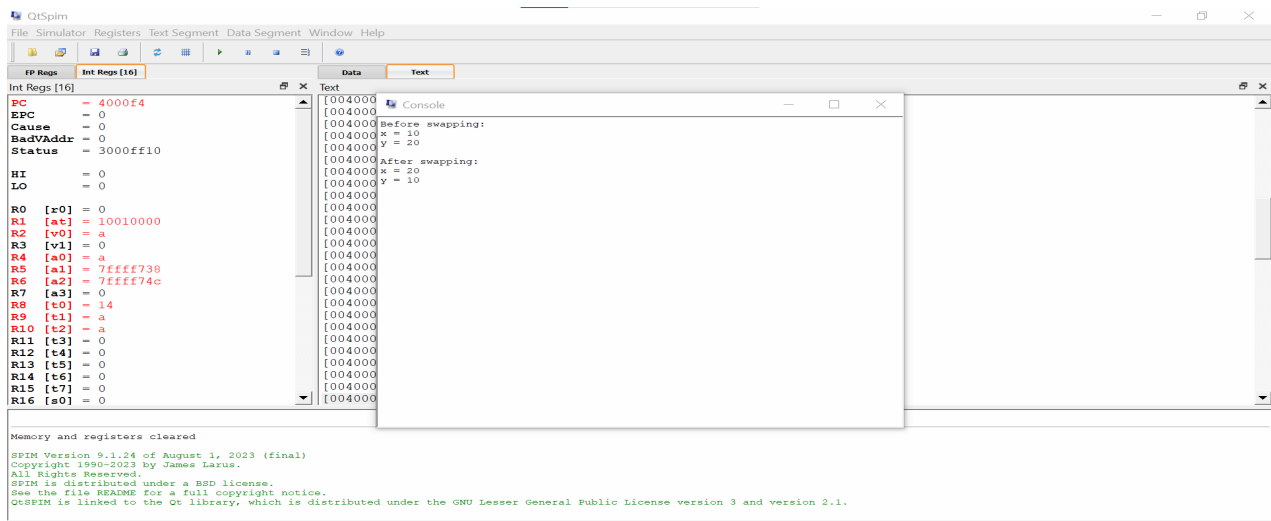
1. **Save** the return address `$ra` on the stack.
2. **Save** any arguments or local variables (like `$a0`).
3. **Check base case** to stop recursion.
4. **Call itself** using `jal` (jump and link).
5. **Restore** saved values before returning with `jr $ra`.

Factorial Program in MIPS:

- $n! = n \times (n-1)!$
- **Base case:** if $n \leq 1 \rightarrow$ return 1

```
#=====
# Program: Recursive Factorial in MIPS
#=====
```

.data



```
prompt: .ascii "Enter a number: "
```

```
resultMsg: .ascii "Factorial is: "
```

```
.text
```

```
.globl main
```

```
main:
```

```
    # Print prompt
```

```
    li $v0, 4
```

```
    la $a0, prompt
```

```
    syscall
```

```
    # Read integer input
```

```
    li $v0, 5
```

```
    syscall
```

```
    move $a0, $v0    # $a0 = n (argument)
```

```
    # Call factorial function
```

```
    jal factorial
```

```
    move $t0, $v0    # $v0 has result, move to $t0
```

```
    # Print result message
```

```
li $v0, 4
```

```
la $a0, resultMsg
```

```
syscall
```

```
# Print factorial value
```

```
li $v0, 1
```

```
move $a0, $t0
```

```
syscall
```

```
# Exit
```

```
li $v0, 10
```

```
syscall
```

```
#-----
```

```
# factorial function
```

```
# Input: $a0 = n
```

```
# Output: $v0 = factorial(n)
```

```
#-----
```

```
factorial:
```

```
addi $sp, $sp, -8    # Make space on stack
```

```
sw $ra, 4($sp)      # Save return address
```

```
sw $a0, 0($sp)    # Save argument n
```

```
# Base case: if  $n \leq 1$ , return 1
```

```
ble $a0, 1, base_case
```

```
# Recursive case:  $n * \text{factorial}(n-1)$ 
```

```
addi $a0, $a0, -1  #  $n = n - 1$ 
```

```
jal factorial      # call factorial(n-1)
```

```
# On return,  $\$v0 = \text{factorial}(n-1)$ 
```

```
lw $a0, 0($sp)    # restore n
```

```
mul $v0, $a0, $v0  #  $v0 = n * \text{factorial}(n-1)$ 
```

```
j end_factorial
```

```
base_case:
```

```
li $v0, 1          #  $\text{factorial}(0 \text{ or } 1) = 1$ 
```

```
end_factorial:
```

```
lw $ra, 4($sp)     # restore return address
```

```
addi $sp, $sp, 8    # free stack space
```

```
jr $ra             # return to caller
```

Execution:

factorial (5): need factorial (4)

factorial (4): need factorial (3)

factorial (3): need factorial (2)

factorial (2): need factorial (1)

factorial (1): Here's 1

factorial (2): Thanks! $2 * 1 = 2$

factorial (3): Thanks! $3 * 2 = 6$

factorial (4): Thanks! $4 * 6 = 24$

factorial (5): Thanks! $5 * 24 = 120$

Array Declaration

The **.data** directive marks the beginning of the data section (where variables and arrays are stored).

.data

numbers: .word 10, 20, 30, 40, 50

- numbers is the **label** (the array name).
- .word means each element takes **4 bytes**.
- It stores 5 integers: 10, 20, 30, 40, 50.

Address	Value
numbers	10

numbers+4	20
numbers+8	30
numbers+12	40
numbers+16	50

.data

array: .space 40 **# reserves 40 bytes (enough for 10 words) reserve space (without initializing):**

Accessing Array Elements:

In MIPS, arrays are accessed using **base address + offset**.

.text

main:

la \$t0, numbers **# load base address of array into \$t0**

lw \$t1, 0(\$t0) **# load first element (numbers[0])**

lw \$t2, 4(\$t0) **# load second element (numbers[1])**

lw \$t3, 8(\$t0) **# load third element (numbers[2])**

Q.1.print all elements of an array of 5 integers.

.data

numbers: .word 10, 20, 30, 40, 50 **# array of 5 integers**

size: .word 5 **# number of elements**

msg: .asciiz "Array elements:\n"

.text

main:

print message

li \$v0, 4 # syscall for print_string

la \$a0, msg

syscall

la \$t0, numbers # base address of array

lw \$t1, size # number of elements now t1=5 (array length)

li \$t2, 0 # set index register \$t2 to 0. We'll use \$t2 as the loop index

loop:

beq \$t2, \$t1, exit # branch if equal compare if index == size then exit

sll \$t3, \$t2, 2 # \$t3 = \$t2 << 2 → multiply index by 4 (because each word = 4 bytes).

add \$t4, \$t0, \$t3 # add base address \$t0 + offset \$t3 → address of
numbers[index] put into \$t4.

lw \$t5, 0(\$t4) # load numbers[index]

print integer

li \$v0, 1 # syscall for print_int

move \$a0, \$t5

syscall

print a space

```
li $v0, 11      # syscall for print_char
```

```
li $a0, 32      # ASCII for space ' '
```

```
syscall
```

```
addi $t2, $t2, 1    # index++
```

```
j loop
```

exit:

```
li $v0, 10        # exit syscall
```

```
syscall
```

Terminologies used in above code:

beq \$register1, \$register2, label

- If (\$register1 == \$register2), then **jump to label**.
- Otherwise, **continue** with next instruction

Registers used:

- \$t0 = base address of array
- \$t1 = array size (length)
- \$t2 = loop index
- \$t3 = byte offset (index*4)
- \$t4 = address of current element
- \$t5 = value of current element
- \$v0 = syscall code (convention)
- \$a0 = argument to syscall (string address, int to print, or char)

Why sll by 2? Because each. word is 4 bytes, so offset in bytes = index * 4.

move is a pseudo instruction. If your simulator doesn't accept pseudo instructions, `move $a0,$t5` becomes `add $a0,$t5,$zero`.

Make sure your simulator settings allow syscalls and pseudo-instructions. In QtSpim enable pseudo instructions (usually on by default).

Output format: The code prints:

Array elements:
10 20 30 40 50

Lab Tasks:

1. Write a MIPS program to subtract and multiply two numbers and print all results.
2. Write a program to calculate the **factorial** of a number entered by the user.
Example: Input 5 → Output 120
3. Declare a variable number initialized with value 7. Load it into a register, add 5 to it, store the new value back into memory, and print the updated value.
4. Declare a variable `num = 4` and another variable `square = 0`.
5. Calculate the square of `num` (`num × num`), store the result in `square`, and print it.
6. Write a MIPS program that asks the user for a number `n`, and calculates the sum of numbers from 1 to `n`.
7. Write a MIPS program that takes two integers from the user and prints which one is larger.
8. Write a program that reads a word from the user and prints it in reverse order.